

Repaso de Python

Este documento resume los conceptos principales de la presentación.

Variables y tipos de datos

Las variables permiten almacenar información. Python utiliza tipado dinámico. Los tipos más comunes son str, int, float, bool, list y dict. Es importante seleccionar el tipo adecuado según la información que se desea almacenar.

Tipo Genérico	Nombre	Descripción
Texto	str	Valores de tipo texto (strings) para representar cadenas de caracteres
Numérico	int	Representan números enteros
Numérico	float	Representan números con punto decimal.
Booleano	bool	Representan valores de verdadero o falso.
Lista	list	Tipo de dato que permite almacenar múltiples valores en una sola variable.
Diccionario	dict	Almacena información en pares clave–valor.

Operadores

Los operadores aritméticos realizan cálculos (+, -, *, /, %, //, **). Los operadores de comparación permiten comparar valores (==, !=, >, <, >=, <=) y los lógicos (and, or, not) combinan condiciones.

```
>>> nombre = "Daniel"
>>> Nombre = "daniel"
```

Operador	Nombre	Descripción
+	Suma	Suma dos valores numéricos
-	Resta	Resta dos valores numéricos
*	Multiplicación	Multiplica dos valores numéricos
/	División	Divide dos valores, brindando el resultado con decimales
%	Módulo	Divide dos valores numéricos y el resultado corresponde al residuo del valor obtenido
//	División entera	Divide dos valores numéricos y el resultado corresponde a la parte entera del valor obtenido
**	Exponenciación	Eleva el primer operador a la potencia indicada por el segundo operador

Operador	Descripción
==	Si los operadores son iguales
!=	Si los operadores son diferentes
>	Si el primer operador es mayor al segundo
>=	Si el primer operador es mayor o igual al segundo
<	Si el primer operador es menor al segundo
<=	Si el primer operador es menor o igual al segundo

Operador	Descripción
and	Devuelve True si todas las expresiones son verdaderas
or	Devuelve False si todas las expresiones son falsas
not	Invierte el valor de la expresión

Entrada y salida

La función `input()` recibe información del usuario y `print()` muestra resultados. Generalmente es necesario convertir el texto recibido a un tipo numérico usando `int()` o `float()`.

Estructuras de control

Las estructuras `if`, `for` y `while` permiten controlar el flujo del programa. La instrucción `break` finaliza un ciclo cuando se cumple una condición específica.

 <pre> if condición: linea 1 linea 2 linea 3 luego del if linea 4 luego del if </pre>	 <pre> if condición1: linea 1 linea 2 elif condición2: linea 1 linea 2 else: linea 1 linea 2 linea luego del if 1 linea luego del if 2 </pre>
--	--

```
while condición:
    linea 1
    linea 2
linea 1
linea 2

for variable in range(inicio, fin, incremento):
    linea 1
    linea 2
    linea 3
linea 1
linea 2

numero = 0
while True:
    if(numero == 5):
        break
    print(numero)
    numero += 1
```

Archivos

Python puede abrir archivos para leer o escribir utilizando `open()`. Los modos `r`, `w`, `a` y `r+` determinan la operación que se realizará. Siempre es recomendable cerrar el archivo.

Modo	Significado
r	Solo lectura
w	Solo escritura, borrando los contenidos previos
a	Solo escritura, empezando a escribir al final del archivo
r+	Lectura y escritura

```
archivo = open(nombre_archivo, modo)

# Lee n caracteres del archivo
caracter = archivo.read(n)

# Lee una línea del archivo
linea = archivo.readline()

# Lee todo el archivo
texto = archivo.read()

archivo.close()
```

```
archivo = open(nombre_archivo, modo)

archivo.write(mensaje)

archivo.close()
```

```
archivo = open(nombre_archivo, modo)

# Ver índice actual
print(f"Vamos por el carácter {archivo.tell()}")

# Saltar al carácter n
archivo.seek(n)

archivo.close()
```

Caracteres especiales

Código	Significado
\n	Nueva línea
\t	Tabulación
\"	Comilla simple
\'	Comilla doble
\\	Barra inclinada inversa

Funciones, clases y módulos

Las funciones permiten reutilizar código. Las clases agrupan datos y comportamientos mediante atributos y métodos. El método `__init__` inicializa los objetos. Los módulos permiten organizar el código en diferentes archivos y reutilizarlo.

```
# Definición de una función
def saludar():
    print(f"Hola.\nBienvenido al curso.")

# Llamado de la función
saludar()
```

```
# Definición de una función
def suma(a, b):
    resultado = a + b
    # Devuelve un valor
    return resultado

# Llamado de la función
a = 5
b = 3
print(suma(a, b))
```

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        print(f"Hola, soy {self.nombre}")
```

```
personal = Persona("Ana", 25)
personal.saludar()

print(personal.nombre)
```

A code editor window with a light gray background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in a monospaced font with syntax highlighting: 'import' is purple, 'utilidades' is blue, 'sumar' is blue, and numbers and parentheses are black. The code demonstrates two ways to use a module: importing the entire module and then accessing a function, and importing a specific function directly.

```
import utilidades

resultado = utilidades.sumar(2, 3)

from utilidades import sumar

resultado = sumar(2, 3)
```

Actividad 1: Registro de Libros

Desarrolle un programa en Python que permita gestionar un catálogo de libros aplicando los conceptos de clases, objetos y módulos.

El programa debe permitir:

- Registrar la información de varios libros.
- Crear una clase llamada Libro para representar cada libro.
- Solicitar y almacenar la siguiente información:
 - Título
 - Autor
 - Año de publicación
- Implementar un método que permita mostrar la información de un libro.
- Ubicar la clase Libro dentro de una carpeta llamada entidades.
- Crear el archivo `__init__.py` dentro de la carpeta entidades.
- Importar la clase Libro desde el archivo principal del programa.
- Crear al menos tres objetos de la clase Libro.
- Mostrar la información de todos los libros registrados.